

國立臺南大學資訊工程學系
資工三「多媒體系統」課程
第三次作業

班級：資工三

姓名：莊旻芳

學號：S11259019

老師：林朝興

中華民國 115 年 5 月 9 日

一、 作業目的

本作業旨在透過實作 Motion Vector Search，了解影像序列中相鄰影格之間的運動關係，並模擬影片壓縮中常見的 Motion Estimation 與 Motion Compensation 流程。藉由將影像切割為固定大小的 macroblock，並在參考影格中搜尋最相似的區塊，可建立各區塊的 motion vector，進而重建 motion compensated frame。

此外，本作業透過比較不同搜尋策略，包括 Exhaustive Search、Logarithmic Search、Hierarchical Search，以及選作的 Diamond Search，分析各方法在補償品質與搜尋成本之間的差異。實驗中以 SAD、PSNR 與 Average Search Points 作為評估指標，進一步理解不同 motion vector search 方法在效率與品質上的 trade-off。

二、 動作估測與動作補償原理

在連續影像序列中，相鄰影格之間通常具有高度相關性，因此可利用前一幀影像作為 reference frame，推估當前影格中各區塊的位移情形。此過程稱為 Motion Estimation。在 block-based motion estimation 中，會先將當前影格切分為固定大小的 macroblock，例如 16x16，再於參考影格的搜尋範圍內尋找與該區塊最相似的位置，並將其位移量記錄為 motion vector。

當完成所有區塊的 motion estimation 後，即可根據每個區塊的 motion vector，自 reference frame 中擷取對應位置的區塊，重新組合成 motion compensated frame，此過程稱為 Motion Compensation。若 motion vector 估測準確，則重建出的 compensated frame 會與原始 current frame 相當接近，代表該方法具有較佳的補償效果。

本作業採用 SAD (Sum of Absolute Differences) 作為 block matching 的 cost function。其概念為計算目前區塊與候選區塊之間像素差值絕對值的總和，數值越小表示兩個區塊越相似，因此可作為搜尋最佳 matching block 的依據。

1. Exhaustive Search:

此方法會在指定搜尋範圍內檢查所有可能的候選位置，逐一計算 SAD，並選擇 SAD 最小者作為最佳匹配結果。由於搜尋完整，因此通常能得到最佳補償品質，但所需搜尋點數最多，計算成本也最高。

2. Logarithmic Search:

此方法以目前搜尋中心為基準，利用步長 (step size) 進行 9 點搜尋。若最佳點不在中心，則將中心移至最佳點並繼續搜尋；若最佳點位於中心，則將步長減半，直到步長縮小至最小值，再進行局部 refinement。此方法可在保有一定品質的情況下，大幅減少搜尋點數。

3. Hierarchical Search:

此方法利用影像金字塔進行多解析度估測，先在低解析度影像上進行粗略 motion estimation，再將所得 motion vector 放大後帶回高解析度影像中進行局部 refinement。此方法特別適合處理較大位移情形，並能降低直接在原解析度全範圍搜尋的成本。

4. Diamond Search:

此方法使用菱形搜尋樣板進行快速搜尋，通常分為 Large Diamond Search Pattern (LDSP) 與 Small Diamond Search Pattern (SDSP)。先以 LDSP 搜尋較大範圍，若最佳點不在中心則移動中心並重複；當最佳點落於中心時，再以 SDSP 做最後 refinement。由於其搜尋形狀更貼近自然影像中常見的運動分布，因此常具有不錯的效率。

在 motion vector search 中，平均搜尋點數 (Average Search Points) 可定義為各 macroblock 在搜尋過程中實際檢查之候選位置數量的平均值。由於每一個候選位置皆需進行 block matching cost 計算，因此此指標不僅描述了搜尋方法在候選點檢查上的數量特性，也能有效反映其整體計算成本。換言之，平均搜尋點數越高，通常代表搜尋所需的運算量越大；平均搜尋點數越低，則表示方法在維持一定補償效果的前提下，具有較高的搜尋效率，因此可作為評估不同 motion vector search 方法成本的重要依據。

三、 主要公式

1. Motion Vector 定義：

若目前區塊位於 current frame 中座標 (x,y) ，而在 reference frame 中找到最佳匹配區塊座標 $(x + dx, y + dy)$ ，則其 motion vector 可表示為：

$$MV = (dx, dy)$$

其中：

- dx ：水平方向位移
- dy ：垂直方向位移

2. SAD (Sum of Absolute Differences)

SAD 用來衡量 current block 與 reference block 之間的差異，其公式如下：

$$SAD(dx, dy) = \sum \sum |C(x + i, y + j) - R(x + dx + i, y + dy + j)|$$

其中：

- $C(x + i, y + j)$ ：current frame 中 block 內像素值
- $R(x + dx + i, y + dy + j)$ ：reference frame 中候選 block 的像素值
- (dx, dy) ：候選位移量

SAD 越小，表示候選區塊與當前區塊越相似，因此搜尋目標即為找出最小 SAD 對應的位置。

3. MSE (Mean Squared Error)

為評估 compensated frame 與原始 current frame 的差異，可先計算 MSE：

$$MSE = (1 / MN) \sum \sum [I(x, y) - K(x, y)]^2$$

其中：

- $I(x, y)$ ：原始 current frame 像素值
- $K(x, y)$ ：motion compensated frame 像素值

- M, N：影像寬與高

4. PSNR (Peak Signal-to-Noise Ratio)

根據 MSE 可進一步計算 PSNR：

$$PSNR = 10 \log_{10}(255^2 / MSE)$$

PSNR 越高，表示重建結果越接近原始影像，補償品質越好。

5. Average Search Points

在 block matching motion estimation 中，主要運算成本來自 candidate block 的 SAD/MAD 計算。因此，本實作以「搜尋點數 (search points)」表示實際被評估的 candidate positions 數量。為了公平比較不同解析度與不同 frame 的結果，進一步使用：

$$Average Search Points = \frac{Total Search Points}{Number of Macroblocks}$$

表示平均每個 macroblock 所需搜尋的 candidate 數量，用來衡量 motion estimation 的 computational complexity。

四、 實作方法

以 C++ 與 OpenCV 為工具，實作 Block-based Motion Estimation 與 Motion Compensation 流程，包含連續影格讀取、macroblock 分割、Block Matching、Motion Vector Search、Motion Compensation、Motion Vector 視覺化，以及影像品質與搜尋成本分析 (SAD、PSNR 與 Average Search Points)。

程式會對相鄰 frame pair (Frame t-1 → Frame t) 逐 block 進行 motion estimation，利用不同搜尋策略尋找最佳 matching block，並根據 motion vectors 建立 motion compensated frame。最後計算 compensated frame 與原始 current frame 之間的 PSNR 與 SAD，同時統計各方法的平均搜尋點數，以分析不同 motion vector search 方法在補償品質與搜尋成本之間的 trade-off。

開發環境：

項目	內容
作業系統	Windows
編譯器	Visual Studio 2022 MSVC
函式庫	OpenCV 4.12.0
語言標準	C++17
輸入	連續影格灰階影像 (PNG / JPG / BMP)
輸出	Motion compensated frame、motion vector map、PSNR/SAD 統計結果與 CSV 圖表資料

固定參數：

<pre>const int kBlockSize = 16; const int kSearchRange = 7; const int kDefaultFramePairs = 30; const vector<string> kMethods = {"exhaustive", "logarithmic", "hierarchical", "diamond"};</pre>

其中，block size 固定為 16x16，search range 固定為 ± 7 ，並以最多前 30 組相鄰影格作為預設序列分析範圍。除了三種必做方法外，程式亦額外實作 Bonus 的 Diamond Search。

主要 function：

processSequence()	序列多幀模式入口，用於分析前 N 組相鄰影格
processSingleFrame()	單幀模式入口，用於分析指定某個 frame index
runAllMethods()	指定每一種方法給 processFramePair()
processFramePair()	對單一組 frame pair 和指定方法完成 motion estimation(逐個 block 丟進 method)、補償與評估。
exhaustiveSearch()	block-level Full Search，在搜尋範圍內檢查所有 candidate block。
logarithmicSearch()	block-level Logarithmic Search，用 step size 縮小搜尋範圍。

hierarchicalSearch()	block-level Hierarchical Search，先低解析度粗估，再回高解析度 refinement。
diamondSearch()	block-level Diamond Search，先使用 Large Diamond Pattern 搜尋，再以 Small Diamond Pattern refinement。
computeBlockSAD()	計算 current block 與 reference candidate block 的 SAD。
buildCompensatedFrame()	根據 motion vectors 從 reference frame 建立 compensated frame

helper function:

clampInt()	限制數值不要超出範圍
ensureOutputDir()	確認輸出資料夾存在
saveImage()	存圖片
computeMSE()	計算 MSE
computePSNR()	計算 PSNR
hasImageExtension()	判斷是不是圖片副檔名
collectInputImages()	蒐集輸入資料夾中的所有影格圖片並排序。
loadGrayImage()	讀取影像並轉成灰階。
parseInteger()	將字串安全轉成整數。
printUsage()	顯示命令列使用方式。
parseArguments()	解析程式執行參數與模式。
isValidCandidate()	檢查 candidate block 有沒有超出邊界
loadFramePair()	讀入 reference/current 兩張圖
drawMotionVectorMap()	將每個 block 的 motion vector 畫成箭頭圖。

搜尋方法 pseudocode

1. exhaustiveSearch()

```
for each block in current frame
    bestSAD = infinity
    for dy = -searchRange to +searchRange
        for dx = -searchRange to +searchRange
            if candidate block is valid
                compute SAD
                if SAD < bestSAD
                    update bestSAD
                    update best motion vector
return best motion vector
```

2. logarithmicSearch()

```
for each block in current frame
    center = (0, 0)
    step = largest power of 2 not greater than searchRange
    while step >= 1
        evaluate 9 candidate points around current center
        find point with minimum SAD
        if best point is center
            step = step / 2
        else
            move center to best point
    perform local refinement around final center
return best motion vector
```

3. hierarchicalSearch()

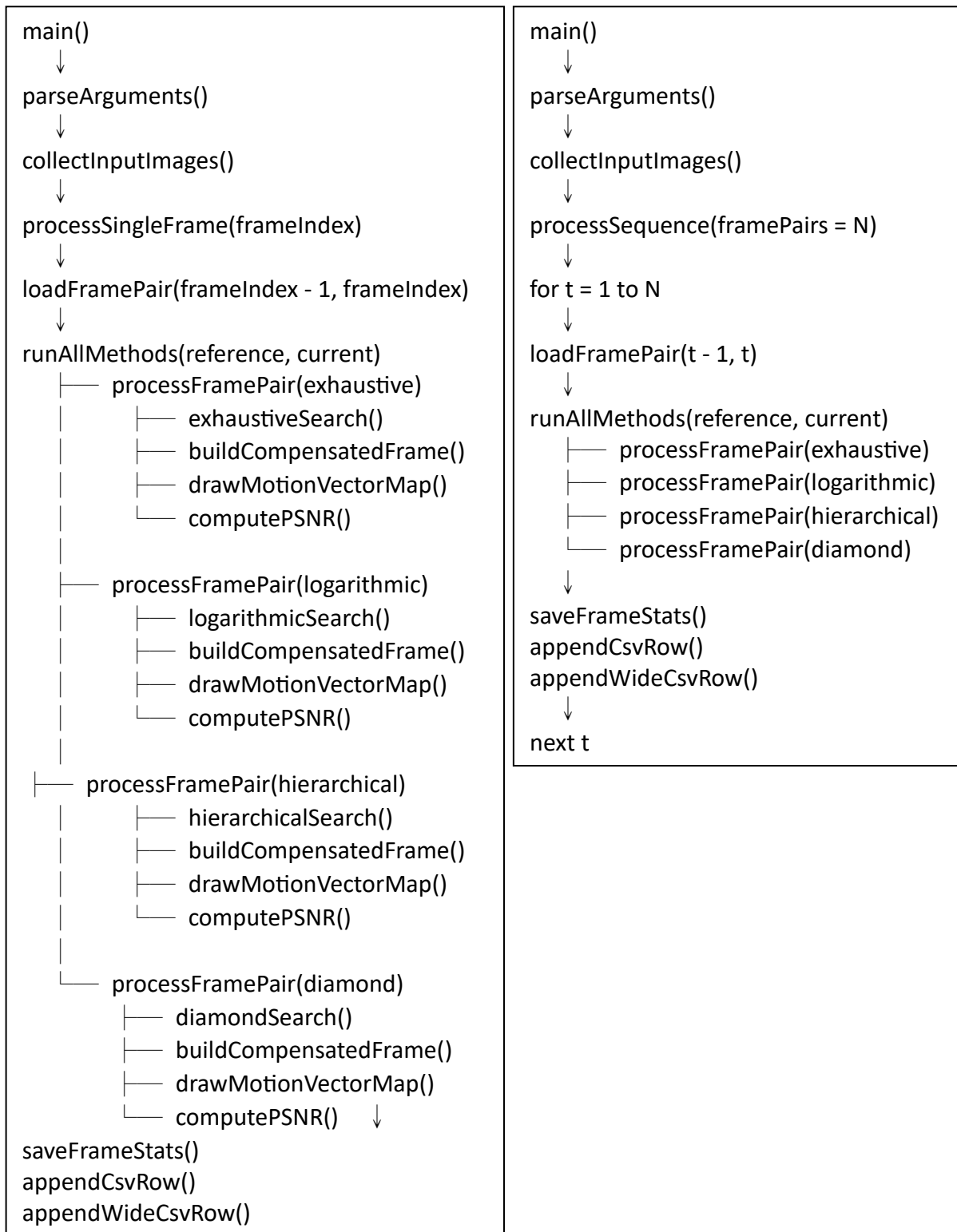
```
for each block in current frame
    build low-resolution images by pyramid downsampling
    perform coarse search in low-resolution level
    scale motion vector to higher-resolution level
    perform local refinement in original-resolution level
return best motion vector
```

4. diamondSearch()

```
for each block in current frame
  center = (0, 0)
  repeat
    evaluate Large Diamond Search Pattern (LDSP)
    find point with minimum SAD
    if best point is not center
      move center to best point
  until best point is center
  evaluate Small Diamond Search Pattern (SDSP)
return best motion vector
```

兩種程式執行模式流程圖

processSingleFrame()/processSequence() 流程圖



五、 實驗結果

影片序列

測試影片名稱	Foreman
來源	Xiph.org Video Test Media
原始檔案格式	Y4M
影像格式	YUV 4:2:0
解析度	CIF 352 x 288
總幀數	300 frames
幀率	約 29.97 fps
實驗輸入格式	逐幀 PNG 影像序列
前處理方式	使用 FFmpeg 將 foreman_cif.y4m 轉為逐幀 PNG 影像序列
單幀比較	第 3 幀或第 15 幀
多幀分析	前 30 組相鄰影格

單幀分析

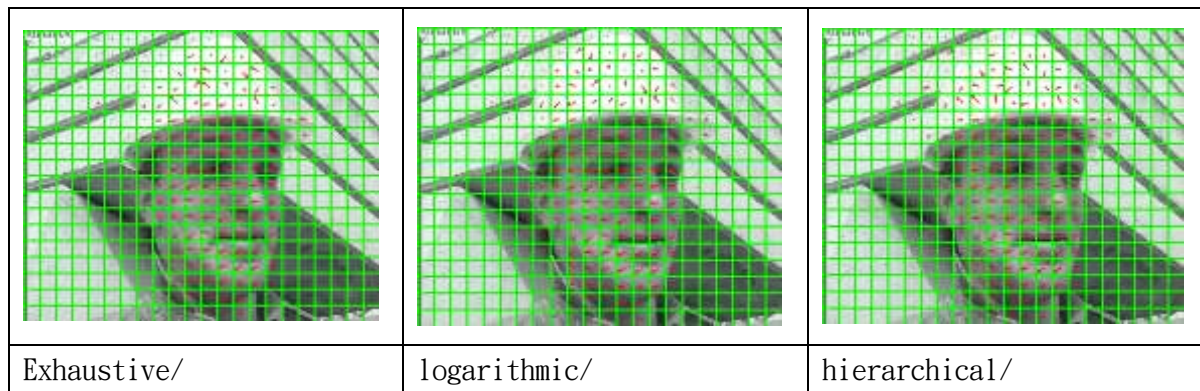
1. compensated_frame_3.png

		
Exhaustive/	logarithmic/	hierarchical/

三種必做方法所重建之影像整體效果相近，畫面皆能維持正常結構，未出現明顯錯位或嚴重失真情形，表示各方法皆能在此幀上有效完成 motion compensation。不過，放大觀察後仍可看出部分區域存在 macroblock 的邊界痕跡，顯示 block-based motion estimation 在區塊交界處仍可能產生局部不連續現象，這也是區塊式補償方法正常的現象。

motion_vector_3.png

2. motion_vector_3.png



三種必做方法在整體運動方向上的估測結果大致一致，皆能反映此幀中的主要運動趨勢。以人物臉部區域為例，可明顯看出向量整體呈現向左的偏移，對應到人物頭部向左轉動的畫面變化。相較之下，背景區域的 motion vector 普遍較短，表示其位移量相對較小，顯示本幀中的主要運動主要集中於人物前景，而背景變化較有限。

然而，在帽子的區域中，不同搜尋方法所得到的 motion vector 並不完全相同，部分區塊的向量方向甚至呈現分散或不規則排列。此現象可能與該區域亮度分布相近、局部紋理特徵不明顯有關，使多個候選區塊之間的 SAD 差異較小，因此不同方法可能選擇不同的 matching position。

雖然帽子區域的 motion vector 分布在不同方法之間存在差異，但對應的 motion compensated frame 在視覺上仍相當接近，未出現明顯補償失敗。這表示在某些紋理較少或相似度較高的區域中，即使不同方法選擇了不同的 motion vector，仍可能得到相近的補償結果。因此，在分析 motion estimation 結果時，除了觀察 motion vector 圖之外，也需搭配 compensated frame 與數值指標一同比較，才能較完整評估各搜尋方法的實際表現。

3. 數值結果比較:

Method	PSNR(dB)	Total SAD	Average Search Point
Exhaustive Search	32.6997 dB	329435	204.2828
Logarithmic Search	32.6072 dB	333196	37.2828
Hierarchical Search	32.6150 dB	334098	66.7980

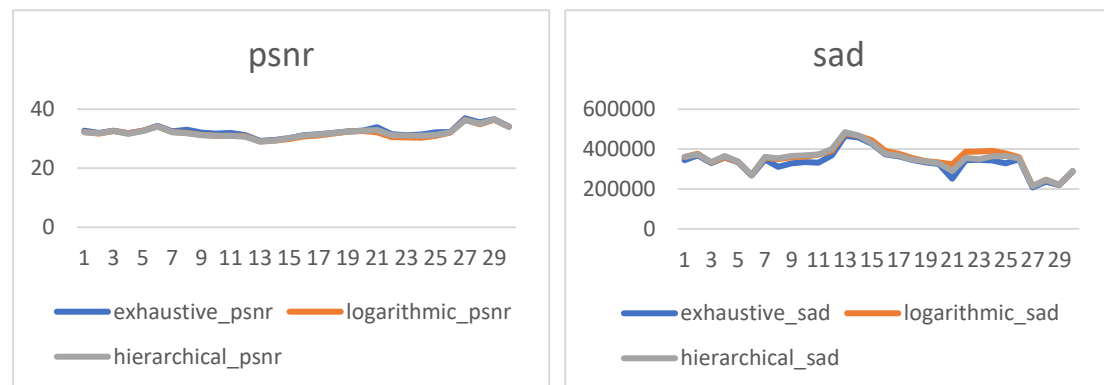
由單幀數值結果可知，Exhaustive Search 具有最高的 PSNR 與最低的 SAD，顯示其補償品質最佳；然而，相較於 Logarithmic Search 與 Hierarchical Search，其平均搜尋點數大幅增加，計算成本明顯較高。雖然 Exhaustive

Search 在數值上略優於其他方法，但差異幅度並不大，且從補償畫面觀察也未出現明顯視覺差距。

綜合 motion vector 圖與補償結果推測，這些 SAD 與 PSNR 的差異可能主要來自帽子等局部區域的 block matching 結果差異。然而，這類局部改善對整體畫面品質的提升有限，與搜尋點數大幅增加所帶來的成本相比，成本效益並不突出。換言之，Exhaustive Search 雖能提供最佳結果，但其補償品質提升幅度遠小於搜尋成本的增加幅度，因此在效率與品質兼顧的情況下，未必是最具實用性的選擇。

相較之下，Logarithmic Search 與 Hierarchical Search 雖在 PSNR 與 SAD 上略遜於 Exhaustive Search，但所需搜尋點數顯著降低，其中 Logarithmic Search 的搜尋成本最低且品質接近基準法，因此在本幀結果中展現出較佳的成本與品質折衷特性。

前 30 幀曲線圖分析

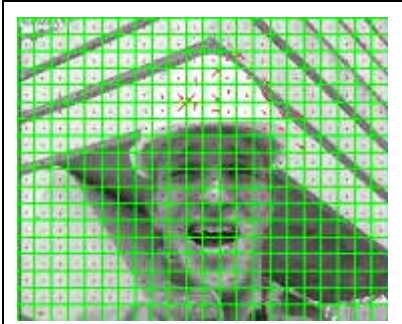
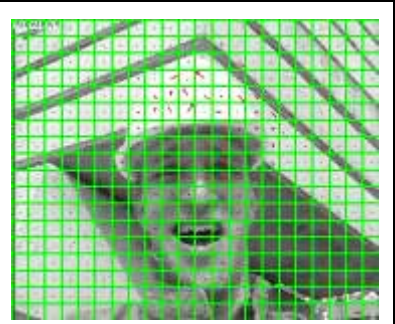


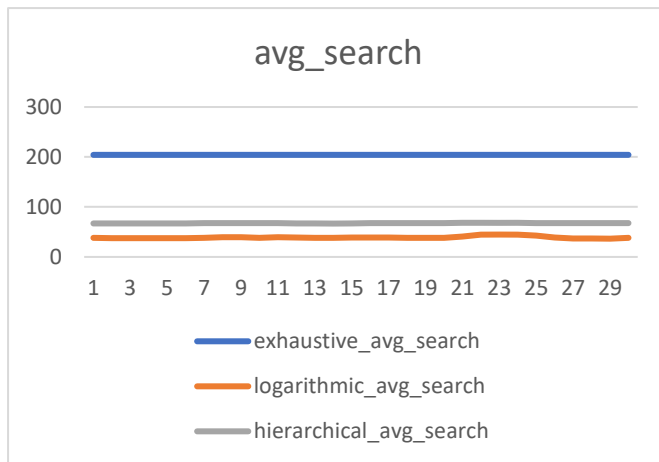
由前 30 幀的 PSNR 與 SAD 曲線可觀察到，三種必做方法在整體變化趨勢上大致一致，表示它們對時間序列中主要畫面變化的補償結果具有相近的整體趨勢。其中，Exhaustive Search 在所有影格中皆具有最高的 PSNR，同時也具有最低的 SAD，符合其作為完整搜尋基準法的特性。Hierarchical Search 的曲線多數情況下與 Exhaustive Search 相當接近，而 Logarithmic Search 在部分區段則可觀察到 PSNR 略低、SAD 略高的情形，顯示其在降低搜尋點數的同時，於某些影格中可能產生較明顯的補償誤差。

整體而言，PSNR 與 SAD 的逐幀變化具有一致的對應關係：Exhaustive Search 始終具有最佳補償品質，Hierarchical Search 多數情況下可維持與基準法接近的表現，而 Logarithmic Search 在部分影格中則會出現較明顯的品質下降。

進一步檢視 Logarithmic Search 在較低 PSNR 且較高 SAD 影格中的 motion vector map，可發現在第 23 幀附近，部分背景區域出現較多方向一致且向量偏長的 motion vector，而此現象在 Exhaustive Search 與 Hierarchical Search 中相對不明顯。推測其原因可能與 Logarithmic Search 的搜尋方式有關：由於其採用固定步長的候選點檢查，並依據當前最佳點逐步移動搜尋中心，在背景區域具有重複紋理、亮度分布相近，且背景位移較前期影格更明顯時，較容易受到搜尋路徑與局部最佳解影響。當多個候選區塊之間的 SAD 差異不大時，演算法可能沿著某一方向持續選擇局部較佳點，進而造成部分區塊的 motion vector 呈現同方向且偏長的結果，這也可能是該幀 PSNR 略低且 SAD 略高的原因之一。

motion_vector_23.png

		
Exhaustive/	logarithmic/	hierarchical/

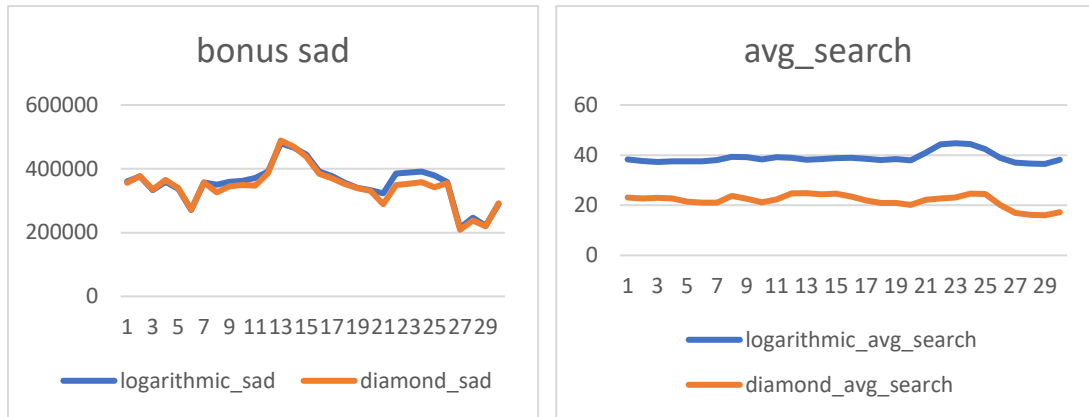


由平均搜尋點數結果可觀察到，Exhaustive Search 在所有影格中的平均搜尋點數皆固定為 204.2828，表示其對每個 macroblock 皆進行完整搜尋，因此搜尋成本固定且最高。相較之下，Hierarchical Search 與 Logarithmic Search 的平均搜尋點數則存在非常些微的波動，主要原因之一是當最佳搜尋位置或候選點接近影像邊界時，部分搜尋點會因超出合法範圍而無法納入計算，因此實際搜尋點數會略為減少。

就數值範圍而言，Logarithmic Search 的平均搜尋點數約介於 36.4899 至 44.7449 之間，為三種方法中最低；Hierarchical Search 則約介於 66.7121 至 68.1490 之間，整體高於 Logarithmic Search，但仍顯著低於 Exhaustive Search。此結果顯示，雖然快速搜尋法的實際搜尋點數會因邊界條件與搜尋過程而產生非常小幅度的變化，但整體而言仍可大幅降低搜尋成本，其中 Logarithmic Search 的成本下降最為明顯，Hierarchical Search 則在成本與品質之間呈現較為折衷的特性。

六、 Bonus

將 Diamond Search 與 Logarithmic Search 進行比較，主要原因在於兩者皆屬於快速搜尋法，其共同目標都是在不進行完整搜尋的情況下，僅對搜尋中心附近的特定候選點進行取樣，盡可能降低搜尋點數，同時維持接近基準法的補償品質。相較於 Exhaustive Search 著重於完整搜尋，Diamond Search 與 Logarithmic Search 更適合從搜尋效率與品質折衷的角度進行比較。



由 SAD 曲線可觀察到，Diamond Search 與 Logarithmic Search 在前 30 幀中的整體變化趨勢相當接近，表示兩者在補償誤差上的表現大致相似。部分影格中，Diamond Search 的 SAD 甚至略低於 Logarithmic Search，顯示其並未因降低搜尋點數而明顯犧牲補償品質。

另一方面，由平均搜尋點數曲線可明顯看出，Diamond Search 在所有影格中的搜尋點數皆低於 Logarithmic Search，且差距相當穩定。這表示 Diamond Search 能以更少的候選點檢查次數完成 motion vector search，因此在搜尋效率上具有明顯優勢。

在搜尋點數皆低於 Logarithmic Search 的情況下，Diamond Search 的 SAD 仍與其相當接近，且部分影格甚至更低。此結果與 Diamond Search 的設計概念一致，顯示其可透過較少的搜尋點數維持接近的補償品質。換言之，菱形搜尋樣板能更集中地檢查搜尋中心附近較可能出現最佳匹配的位置，因此在自然影像常見的局部連續運動條件下，具有良好的搜尋效率。

綜合而言，Diamond Search 在本實驗中能以較低的搜尋成本維持與 Logarithmic Search 接近的補償品質，顯示其在快速搜尋法中具有良好的成本效益。若從品質與效率的折衷角度來看，Diamond Search 在本次測試序列中展現出較高的搜尋效率，且品質損失並不明顯，因此具有作為快速搜尋法延伸方法的價值。

七、 結論

Exhaustive Search 的特性是補償品質最佳且表現最穩定，在所有影格中皆具有最高的 PSNR 與最低的 SAD，因此適合作為品質比較的基準方法。然而，其平均搜尋點數固定且最高，代表搜尋成本最大，計算效率也最低。

Logarithmic Search 的主要特性是搜尋成本最低，平均搜尋點數明顯小於另外兩種方法，因此在速度與效率上具有最大優勢。但其補償品質在部分影格中會略低於其他方法，表示當畫面內容較複雜、背景紋理重複或局部區塊相似度較高時，較容易受到搜尋路徑與局部最佳解影響，進而造成 motion vector 偏差與補償品質下降。因此，Logarithmic Search 較適合用於重視搜尋效率、且能接受些微品質損失的情境。

Hierarchical Search 在整體表現上較為平衡。其補償品質多數情況下接近 Exhaustive Search，而搜尋成本又明顯低於完整搜尋，顯示其在品質與效率之間具有較佳折衷。當畫面位移較大或運動較明顯時，多解析度的 coarse-to-fine 搜尋方式有助於降低直接完整搜尋的成本，因此 Hierarchical Search 較適合用於希望兼顧品質與運算成本的情境。

從補償品質較低的影格來看，快速搜尋法的限制主要出現在局部紋理不明顯、背景具有重複結構或候選區塊之間相似度過高的區域。在這些情況下，不同方法即使得到不同的 motion vector，也可能產生相近的補償畫面；但對 Logarithmic Search 而言，搜尋路徑限制更可能導致部分區塊出現方向一致且偏長的向量，進而影響整體補償品質。

整體而言，Exhaustive Search 最適合作為最佳品質基準，Logarithmic Search 最適合用於重視速度與搜尋效率的情境，而 Hierarchical Search 則在品質與成本之間展現出較佳平衡性，屬於整體較具實用性的搜尋策略。

此外，Diamond Search 的結果顯示，其在搜尋點數低於 Logarithmic Search 的情況下，仍能維持接近的補償品質，顯示其快速搜尋樣板設計具有良好的成本效益。由於自然影像中的運動通常具有局部連續性，最佳匹配位置多半位於搜尋中心附近，因此 Diamond Search 透過菱形樣板集中檢查周圍較可能出現最佳匹配的位置，能以較少搜尋點數完成估測。這也是其在自然影像運動估測中可能具有較佳效率的主要原因。

八、 參考資料

Foreman : <https://media.xiph.org/video/derf/>

OpenCV Documentation: <https://docs.opencv.org>